

Complete the following 11 questions and be sure to show all of your work

Part 1: RISC-V RV32I Instruction Decoding

1. (8 points) The 32-bit instruction below is encoded in hex as 0x007302B3 (binary: 0000 0000 0111 0011 0000 0010 1011 0011). Identify the instruction format, break it into its fields, and write the equivalent RISC-V assembly instruction.
2. (8 points) Encode the instruction `addi x10, x11, -5` into its 32-bit binary representation and give the result in hex. Show the value of each field.
3. (8 points) The instruction 0x0084A623 is given in hex. Identify the instruction format, decode its fields, and write the equivalent assembly instruction.
4. (8 points) The instruction 0xFEC12703 is given in hex. Identify the instruction format, decode its fields (note the immediate is negative), and write the equivalent assembly instruction.
5. (8 points) (a) (4 points) Decode the instruction 0xFE321C23 into assembly.
(b) (4 points) Encode `sub x18, x19, x20` into 32-bit binary and hex.

Part 2: IEEE-754 Floating-Point Conversion

6. (8 points) Convert the decimal value **-22.5** to IEEE-754 **single precision**. Show the sign, exponent, and mantissa bits, then give the final answer in hex.

7. (8 points) Convert the IEEE-754 **single precision** hex value **0xC1500000** back to decimal. Show your work (sign, exponent, mantissa, and the reconstructed value).

8. (8 points) Convert the decimal value **0.1** to IEEE-754 **single precision**. (Note: 0.1 has no exact binary representation — round to 23 mantissa bits and note that the result is an approximation.)

9. (8 points) Convert the decimal value **100.25** to IEEE-754 **double precision**. Show the sign, exponent, and mantissa bits, then give the final answer in hex.

10. (8 points) Convert the IEEE-754 **double precision** hex value **0x4059000000000000** back to decimal.

Part 3: Conceptual Questions

11. (20 points) Answer the following:

- (a) (7 points) What is the purpose of the Program Counter (PC) in a RISC-V processor? How does its value change during normal sequential instruction execution versus during a branch or jump instruction?

- (b) (7 points) Explain how the stack is used during program execution, particularly during function calls. What kinds of data are typically stored on it, and in what order are they added and removed?

- (c) (6 points) What is the purpose of the stack pointer (register `x2` / `sp` in RISC-V)? How does its value change when data is pushed onto or popped off the stack?